GU Tech - Al Ghazali University

Final Exam Spring 2025

Department of Computer Science

Problem Solving & Programming Fundamentals (Lab)

25 August 2025 10:00 am - 01:00 pm Total marks: 55

Instructions

- Attempt all questions. There are 3 questions for a total of 55 marks.
- This is an open book exam; you may consult your lab manuals but peer discussions and Internet access are not allowed.
- You have been provided with starter code along with this question paper. The code is also provided *within* this paper. Be advised that copying from this PDF may ruin code formatting.
  - You may use any indentation style but be consistent.
  - Do **not** modify or remove:
    - the function prototypes.
    - the print statements within the functions.
    - the `if (argc > 1)` block in the `main` function (if present).
- Write clean code. Dirty hacks are grounds for marks deduction. They include but are not limited to premature `return` statements in `void` functions, unnecessarily breaking out of `for` loops, etc.

Questions

1. Given a matrix, print it in a snake pattern. For example, the following matrix (15)

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

is printed as

$$\begin{matrix} 1 & 2 & 3 & 4 \\ 8 & 7 & 6 & 5 \\ 9 & 10 & 11 & 12 \\ 16 & 15 & 14 & 13 \end{matrix}$$

Do not forget to free the dynamically allocated memory at the end of your program in `main()`.

```c
#include <stdio.h>
#include <stdlib.h>

void printSnake(int** matrix, int noOfRows, int noOfCols)
{
        //Student code goes here.
}

int main()
{
    int noOfRows = 4, noOfCols = 4;
    int arr[] = {1, 2, 3, 4,
                 5, 6, 7, 8,
                 9, 10, 11, 12,
                 13, 14, 15, 16};

    int** matrix = (int**) malloc(noOfRows * sizeof(int*));
    for (int i = 0; i < noOfRows; i++)
    {
        matrix[i] = (int*) malloc (noOfCols * sizeof(int));
    }

    for (int i = 0; i < noOfRows; i++)
    {
        for (int j = 0; j < noOfCols; j++)
        {
            matrix[i][j] = arr[i * noOfCols + j];
        }
    }

    printSnake(matrix, 4, 4);

    //Do not forget to free the dynamically allocated memory.

    return 0;
}
```

2. Complete the function `bool areAnagrams(char* strOne, char* strTwo)` that checks if two given strings are anagrams of each other. According to Google, an anagram is: (20)

> A word, phrase, or name formed by rearranging the letters of another, such as spar, formed from rasp.

In other words, two strings are anagrams if they contain the same characters in the same frequency but possibly in a different order.

For example, `"listen"` and `"silent"` are anagrams of each other.

For the sake of convenience, you may assume that anagrams are case sensitive, so that `"Listen"` and `"Silent"` are not anagrams.

```c
#include <stdio.h>
#include <stdbool.h>

int getStringLength(char* str);
bool areAnagrams(char* strOne, char* strTwo)
{
    return true;
}

int main()
{
    char strOne[] = "silent\0";
    char strTwo[] = "listen\0";

    if (areAnagrams(strOne, strTwo))
    {
        printf("%s and %s are anagrams.\n", strOne, strTwo);
    }

    return 0;
}
```

3. Implement the necessary helper functions and complete the function `char* add(char* numOne, char* numTwo, char* result, int stringSize)` that performs addition of two non-negative integers represented as strings. The result should also be stored as a string. Assume that array size for all `char` arrays is 20 characters including the null character. (20)

Hint: You will need a helper function to pad leading zeros so that `char str[20] = "14\0";` becomes `char str[20] = "00000000000000014\0";`.

Your program should handle all cases. Some examples have been given below for convenience.

Example 1:

```
char smallWithoutCarryOne[20] = "14\0";
char smallWithoutCarryTwo[20] = "14\0";
```

	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	4
+	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	4
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	8

Example 2:

```
char smallWithCarryOne[20] = "15\0";
char smallWithCarryTwo[20] = "15\0";
```

	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	5
+	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	5
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	0

Example 3: Large number with carry

```
char largeOne[20] = "9999999999999999\0";
char largeTwo[20] = "9999999999999999\0";
```

	0	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9
+	0	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9
	1	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	8

Example 4: Large number with carry and overflow. The overflowed digit (in red) is discarded because there is no space to store it.

```
char largeOverFlowOne[20] = "99999999999999999\0";
char largeOverFlowTwo[20] = "99999999999999999\0";
```

	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9
+	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9
1	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	8

```c
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

char* add(char* numOne, char* numTwo, char* result, int stringSize);

int main(void)
{
    char smallWithoutCarryOne[20] = "14\0";
    char smallWithoutCarryTwo[20] = "14\0";
    //Ans: 0000000000000000028

    char smallWithCarryOne[20] = "15\0";
    char smallWithCarryTwo[20] = "15\0";
    //Ans: 0000000000000000030

    char mediumOne[20] = "109876201453\0";
    char mediumTwo[20] = "1203009954\0";
    //Ans: 0000000111079211407

    char largeOne[20] = "9999999999999999\0";
    char largeTwo[20] = "9999999999999999\0";
    //Ans: 1999999999999999998

    char largeOverFlowOne[20] = "99999999999999999\0";
    char largeOverflowTwo[20] = "99999999999999999\0";
    //Actual ans: 19999999999999999998
    //Our ans: 9999999999999999998

    char result[20];

    printf("%s\n", add(smallWithoutCarryOne, smallWithoutCarryTwo,
    result, 20));

    printf("%s\n", add(smallWithCarryOne, smallWithCarryTwo,
    result, 20));

    printf("%s\n", add(mediumOne, mediumTwo, result, 20));

    printf("%s\n", add(largeOne, largeTwo, result, 20));

    printf("%s\n", add(largeOverFlowOne, largeOverflowTwo, result, 20));

    return 0;
}
```

